Memoria Practica 2

GRUPO: 1322

NUMERO DE PAREJA: 8

AUTORES: JORGE PALOMINO Y RAÚL MATELLANO

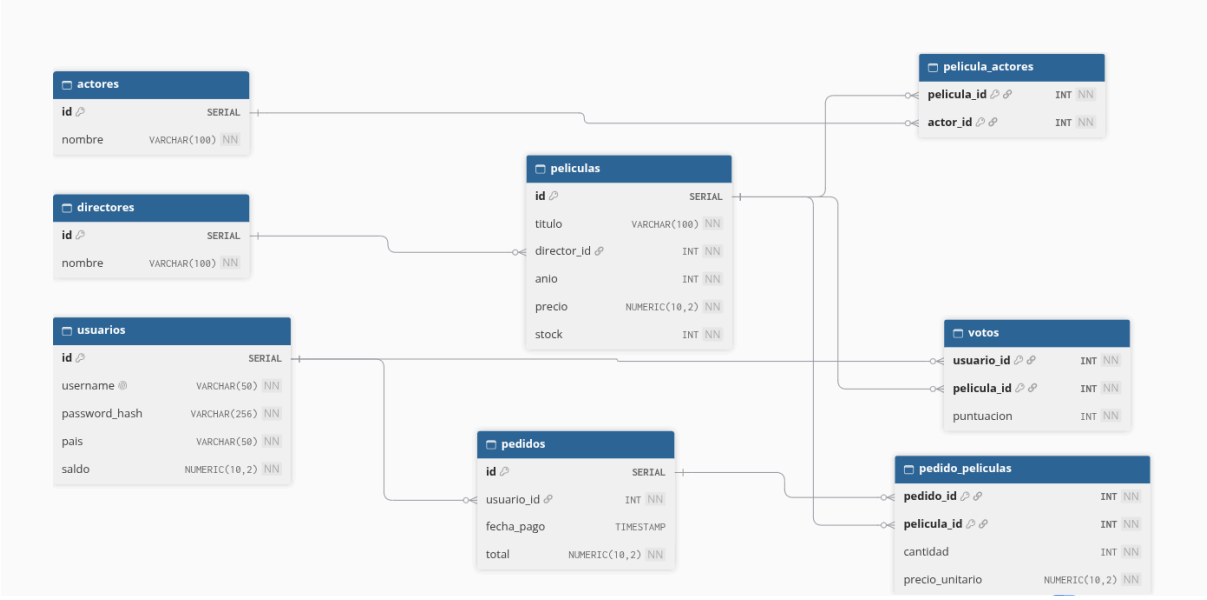
INTRODUCCIÓN Y DISEÑO DE LA BASE DE DATOS

El objetivo principal ha sido diseñar un sistema de videoclub gestionado por una base de datos relacional y desarrollar su API

En lugar de programar una aplicación todo en uno, lo hemos implementado basándose en microservicios, lo que nos aporta modularidad y escalabilidad.

Para el diseño de la base de datos hemos creado 5 tablas principales y tablas intermedias para las relaciones.

Para gestionar el carrito de la compra decidimos que la misma tabla de pedidos actúe como carrito y si el campo fecha_pago es NULL, significa que es el carrito abierto. Además, en la tabla intermedia pedido_peliculas guardamos el precio unitario en el momento en el que el usuario mete la película al carrito, de esta forma, si el precio cambia en un futuro, los pedidos antiguos no se verán afectados.



TRIGGERS Y PROCEDIMIENTOS ALMACENADOS

Para trasladar parte de la lógica de negocio a la base de datos y evitar problemas de sincronización hemos implementado los siguientes triggers:

trg_actualizar_stock: quita el stock al meter una película al carrito y lo devuelve si se elimina

trg_calcular_total_pedido: recalcula automáticamente la suma total de dinero cada vez que se mete o saca algo del pedido.

trg_descontar_saldo: al poner la fecha de pago en el pedido, comprueba si el usuario tiene saldo y se lo resta, si no tiene, bloquea la acción

Para las consultas más complejas, hemos creado procedimientos almacenados:

sp_valoracion_media: calcula la media de las notas que los usuarios han puesto a una película

sp_pagar_carrito: Busca el carrito abierto del usuario y le pone la fecha actual, lo que hace saltar el trigger de cobro

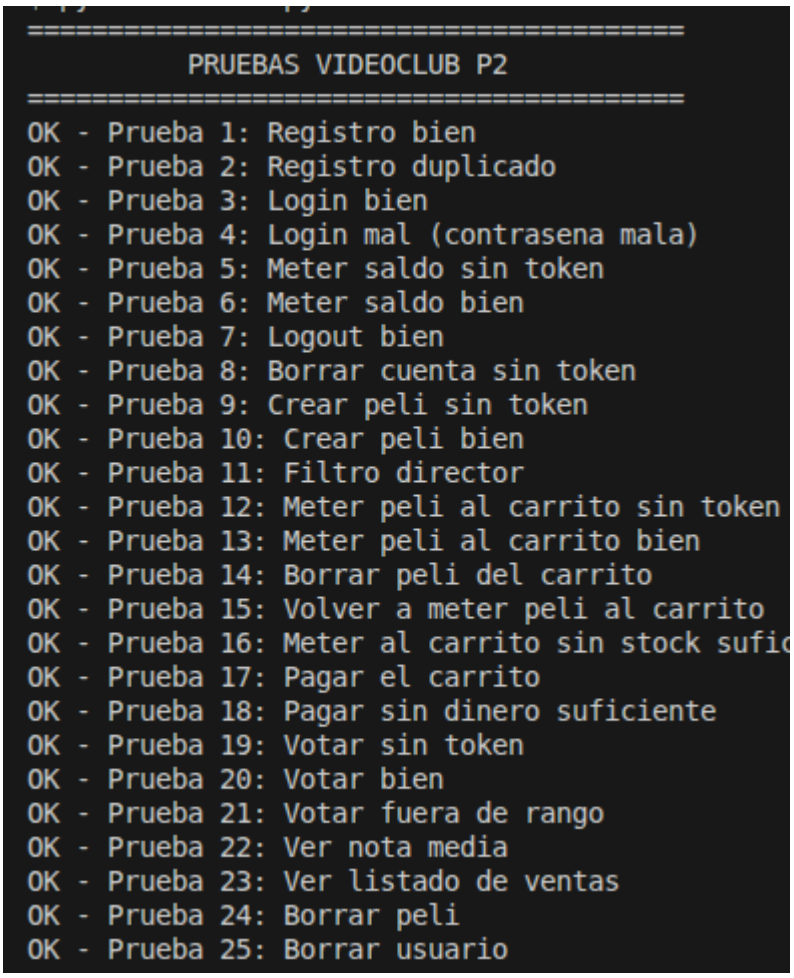
IMPLEMENTACIÓN DE MICROSERVICIOS

Hemos dividido el código en dos servicios independientes. En ambos se ha utilizado Quart, importando Quart y jsonify para levantar el servidor y devolver las respuestas HTTP con la estructura JSON.

Servicio de usuarios “user.py”

Es el servicio que gestiona los registros, inicios de sesión y el saldo del usuario desde el puerto 5050. Para el acceso a los datos hemos utilizado SQLAlchemy, usando la interfaz Core. Para lo relacionado con la autenticación, generamos tokens Bearer localmente con HMAC y SHA-256 usando el username, sin necesidad de guardar el token en la base de datos para no sobrecargar el sistema

Para comprobar el funcionamiento y la integración de ambos microservicios, hemos ejecutado el script de pruebas. Como se puede observar en la siguiente captura, todas las peticiones se ejecutan correctamente:



Servicio principal “api.py”

Este servicio se encarga del catálogo de películas, gestionar el carrito, pagar y sacar listados, funcionando en el puerto 5051. Hemos usado SQLalchemy Core de forma asíncrona. Todas las consultas las hacemos usando text() y execute() con diccionarios de variables

OPTIMIZACIÓN DE CONSULTAS

Para mejorar el rendimiento del endpoint de listado de ventas hemos utilizado la sentencia EXPLAIN ANALYZE sobre nuestra consulta. Al principio, vimos que PostgreSQL realizaba un Seq Scan, leyendo toda la tabla de usuarios y pedidos fila a fila.

Para solucionarlo, hemos creado el script optimizacion.sql donde generamos índices para las columnas por lo que más se filtra: el país del usuario y la fecha de pago del pedido .

Tras aplicar dichos índices, al volver a lanzar el análisis observamos que el tiempo de ejecución de la consulta se reduce a 0.074 ms, mejorando la eficiencia del plan de acceso a los datos de la base de datos:

```
si1=# EXPLAIN ANALYZE SELECT * FROM pedidos p JOIN usuarios u ON p.usuario_id = u.id WHERE u.pais = 'Francia' AND EXTRACT(YEAR FROM p.fecha_pago) = 2026;
                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=0.00..41.74 rows=1 width=804) (actual time=0.037..0.049 rows=2 loops=1)
   Join Filter: (p.usuario_id = u.id)
   Rows Removed by Join Filter: 7
   ->  Seq Scan on usuarios u  (cost=0.00..11.25 rows=1 width=772) (actual time=0.021..0.023 rows=3 loops=1)
         Filter: ((pais)::text = 'Francia'::text)
         Rows Removed by Filter: 3
   ->  Seq Scan on pedidos p  (cost=0.00..30.40 rows=7 width=32) (actual time=0.005..0.006 rows=3 loops=3)
         Filter: (EXTRACT(year FROM fecha_pago) = '2026'::numeric)
         Rows Removed by Filter: 1
 Planning Time: 1.354 ms
 Execution Time: 0.080 ms
(11 rows)

si1=#
```

```
si1=# EXPLAIN ANALYZE SELECT * FROM pedidos p JOIN usuarios u ON p.usuario_id = u.id WHERE u.pais = 'Francia' AND EXTRACT(YEAR FROM p.fecha_pago) = 2026;
                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=0.00..2.15 rows=1 width=804) (actual time=0.037..0.050 rows=2 loops=1)
   Join Filter: (p.usuario_id = u.id)
   Rows Removed by Join Filter: 4
   ->  Seq Scan on pedidos p  (cost=0.00..1.06 rows=1 width=32) (actual time=0.015..0.018 rows=3 loops=1)
         Filter: (EXTRACT(year FROM fecha_pago) = '2026'::numeric)
         Rows Removed by Filter: 1
   ->  Seq Scan on usuarios u  (cost=0.00..1.07 rows=1 width=772) (actual time=0.007..0.008 rows=2 loops=3)
         Filter: ((pais)::text = 'Francia'::text)
         Rows Removed by Filter: 1
 Planning Time: 0.877 ms
 Execution Time: 0.074 ms
(11 rows)

si1=#
```

CONTENEDORES Y DESPLIEGUE

Para el despliegue del sistema hemos decidido usar contenedores Docker. A diferencia de la práctica anterior, además de los servicios, levantamos la base de datos. Para ello hemos usado postgres y montado los scripts schema.sql y populate.sql en la carpeta de inicialización para que la base de datos se cree y se rellene sola al arrancar.

Para gestionar la ejecución hemos utilizado Docker Compose. Para poner todo el sistema en marcha de golpe, basta con escribir el comando docker-compose up -d.